White-Box Testing Report
Money-Poly

Ronith Menneni
2024115018

March 23, 2026

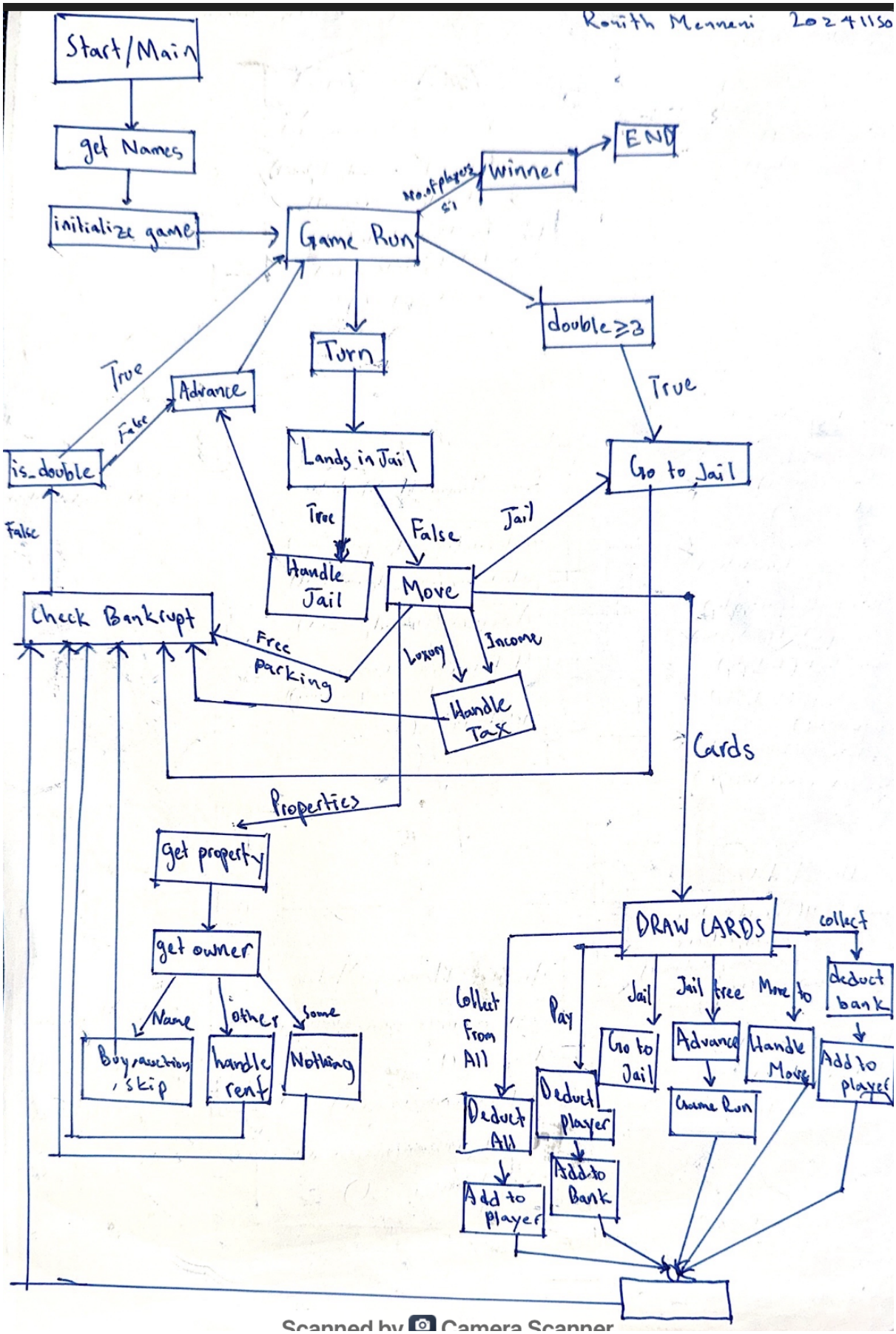
1 Introduction

This report documents the white-box testing process for the Money-Poly game implementation. The project manages players, money, properties, rent payments, and core gameplay mechanics. The objective was to analyze internal control flow, improve code quality using pylint, and design comprehensive white-box test cases using pytest.

2 1.1 Control Flow Graph

2.1 Control Flow Graph (Hand-Drawn)

A hand-drawn control flow graph (CFG) was created for the program. The image is attached below.



Scanned by Camera Scanner

3 1.2 Code Quality Analysis (pylint)

3.1 Iteration Log (from Git history)

Iteration	Summary of Changes (Commit Message)
Iteration 1	Removed trailing newlines, added missing module docstring, and missing function/method docstrings in main.py
Iteration 2	Added missing final newline and changed module docstring to more effective
Iteration 3	Added missing module docstrings, removed redundant math import, switched to relative import for bank starting funds
Iteration 5	Added missing module docstring and simplified boolean comparison for property mortgage check
Iteration 6	Added missing module docstring, reformatted card dictionaries to one key/value per line
Iteration 7	Added missing config module docstring
Iteration 8	Added missing module docstrings, removed unused BOARD$_{SIZEimport, initializeddoublesstreak}$
Iteration 9	Removed unused imports, added module docstring, added targeted pylint disables, simplified GAME OVER banner and menu control flow
Iteration 10	Added module docstring, removed unused import sys, suppressed Player attribute count warning
Iteration 11	Added property module docstring, suppressed Property lint counts, simplified unmortgage flow
Iteration 12	Added ui module docstring and narrowed safe_int_input exception handling

3.2 Final pylint status

All pylint warnings were resolved and code quality requirements were satisfied.

4 1.3 White-Box Test Cases

4.1 Test Strategy

White-box tests were designed to cover:

- All branches (every decision path).
- Key variable states affecting logic (e.g., jail status, mortgage status, balance thresholds).
- Edge cases (e.g., zero values, large inputs, wrap-around movement).

4.2 Test Suite

The test suite was implemented in `pytest` and includes 80+ targeted tests. These tests focus on:

- Dice behavior and reset paths.
- Player movement, Go salary logic, and jail state.
- Board tile types and property lookups.
- Card deck cycling and empty-deck handling.
- Bank transactions, loans, and validations.
- Property group ownership logic.
- Game flow rules (rent transfer, property purchase, winner selection, bankruptcy).

4.3 Why These Test Cases Are Needed (Simple Explanations)

Below are representative tests and a plain-language explanation of why each one is required. The complete list is in `tests/whitebox_test_cases.md`.

#	**Test Case**	**Why it is needed**
1	Dice roll totals stay within 2–12	Ensures dice logic is correct and never returns impossible totals.
2	Dice die1 stays within 1–6	Verifies each die uses the correct range.
3	Dice die2 stays within 1–6	Same as above, but for the second die.
4	Dice reset clears doubles streak	Ensures state is reset between games or turns.
5	Player move by 40 lands on Go and collects salary	Confirms Go salary is given when landing exactly on Go.
6	Player move wrap (39 + 2) collects salary	Checks salary is paid when passing Go, not just landing on it.
7	Tile type for Jail is `jail`	Verifies special tile mapping works correctly.
8	Tile type for Go To Jail is `go_to_jail`	Ensures the game sends players to jail correctly.
9	CardDeck draw cycles with a single card	Confirms deck cycling logic works at boundaries.
10	CardDeck cards_remaining on empty returns 0	Prevents division by zero and invalid deck state.

11	Bank collect ignores negative amounts	Prevents bank balance from decreasing due to invalid inputs.
12	Bank loan reduces bank funds	Ensures money conservation when issuing loans.
13	PropertyGroup all_owned_by requires full set	Avoids giving monopoly benefits for partial ownership.
14	Game buy_property allows exact balance	Ensures players can buy when they have exactly enough money.
15	Game pay_rent transfers rent to owner	Ensures rent actually reaches the property owner.
16	Game find_winner picks highest net worth	Avoids declaring the wrong winner.
17	Unmortgage does not clear mortgage if funds are insufficient	Prevents state corruption when a player cannot pay.

4.4 Errors Found and Fixed

The following logic issues were discovered through testing and fixed.

Error #	**Fix Summary**
Error 1	Pay GO salary when passing GO (wrap-around movement).
Error 2	Fix blank tile test positions (removed non-blank tile).
Error 3	Handle empty CardDeck in `cards_remaining`.
Error 4	Ignore non-positive bank collections.
Error 5	Reduce bank funds when issuing loans.
Error 6	Require full property group ownership for bonuses.
Error 7	Add railroad properties to the board data.
Error 8	Pick highest net worth winner (fix min/max).
Error 9	Allow buying property with exact balance.
Error 10	Transfer rent to property owner.
Error 11	Keep mortgage when funds are insufficient to unmortgage.

4.5 Testing Outcome

All pytest test cases pass after the fixes. The final test run reports:

```
pytest -q
...  (all tests passed)
```